



FLOW GUIDE

Claude Code

O Guia Completo para Todos
Integrado com VS Code, Cursor e Terminal

Do assistente de chat ao sistema operacional de desenvolvimento

**ACESSE AGORA A MASTER CLASSE COMPLETA QUE
ACONTECERÁ DIA 25 DE MARÇO ÀS 20 HORAS**
<https://webnar-claude-code-engineer.flowgrammers.tech/>

flowgrammers.com.br

Março de 2026 • Baseado na documentação oficial da Anthropic

Sumário

1. Por Que o Claude Code Muda Tudo
2. O Que o Claude Code Pode Fazer
3. O Workflow do Claude Code
4. Pré-requisitos e Planos
5. Instalação no Terminal (CLI)
6. Integração com VS Code
7. Integração com Cursor
8. Comandos Essenciais e Atalhos
9. Skills e CLAUDE.md — Memória do Projeto
10. Plan Mode — Planejamento Inteligente
11. Técnicas de Prompting
12. Modelos Disponíveis (Sonnet vs Opus)
13. Conecte Tudo com MCP
14. Workflow Combinado: Terminal + IDE
15. Dicas e Boas Práticas
16. Comparação: Claude Code vs Cursor AI
17. Estrutura de Arquivos do Projeto
18. Hierarquia de Memória do CLAUDE.md
19. Skills Avançadas — O Superpoder
20. Hooks — Automação Determinística
21. Permissões e Segurança
22. Arquitetura de 4 Camadas
23. Workflow Diário Recomendado
24. Recursos e Links Úteis

1. Por Que o Claude Code Muda Tudo

O Claude Code transforma a IA de uma ferramenta de chat que você visita em uma camada de sistema operacional que roda por todo o seu fluxo de trabalho. É a diferença entre pedir ajuda a alguém e ter um colega capaz que realmente executa.

Chat Window Tradicional

- Janela de chat limitada
- Limites de upload
- Sessões expiram
- Você faz tudo manualmente

Claude Code

- Acesso total ao sistema de arquivos
- Qualquer arquivo, qualquer tamanho
- Tarefas de horas rodando
- Agentes e AI Teammates

De Assistente a Sistema Operacional

Claude Code transforma IA de uma ferramenta de chat em uma camada que roda por todo o seu workflow. É a diferença entre pedir ajuda e ter um colega que realmente executa.

2. O Que o Claude Code Pode Fazer

Acesso Total ao Sistema de Arquivos

Leia, escreva, crie e organize qualquer arquivo no seu computador. Sem limites de upload, sem restrições.

- Impacto: "Processe 10.000 arquivos em minutos ao invés de horas de upload manual"

Execução de Comandos e Ferramentas

Rode comandos de shell, execute scripts, gerencie Git, instale pacotes — tudo através de linguagem natural.

- Impacto: "Não-programadores rodando comandos bash sem aprender sintaxe"

Conexões MCP (200+ Ferramentas)

Conecte-se a GitHub, Notion, Slack, Jira, bancos de dados, APIs — o Claude trabalha dentro do seu ecossistema existente.

- Impacto: "Uma interface para todo o seu ecossistema de ferramentas"

Trabalho Multi-Agente Autônomo

Sub-agentes trabalham em paralelo, checkpoints permitem voltar atrás, tarefas em background rodam enquanto você trabalha.

- Impacto: "Delegue como se tivesse 5 desenvolvedores juniors"

3. O Workflow do Claude Code

O Claude Code opera em 4 fases que se repetem, criando um ciclo contínuo de produtividade:

1 Analisar • Sintetizar feedback • Pesquisar concorrentes • Extrair insights • Ler e resumir arquivos	2 Planejar • Rascunhar PRDs • Criar roadmaps • Gerar estratégias • Frameworks de decisão	3 Criar • Gerar apresentações • Escrever código • Criar relatórios • Montar dashboards	4 Escalar • Workflows recorrentes • Conectar tools via MCP • Criar skills reusáveis • Agendar tarefas batch
--	---	---	--

Exemplo para Product Managers

Etapa	Resultado
PRD a partir de transcrição de reunião	Documento estruturado pronto
Tickets Jira criados automaticamente	Backlog populado
Resumo enviado no Slack	Equipe atualizada
Dashboard atualizado	Métricas visíveis

4. Pré-requisitos e Planos

Requisitos do Sistema

Requisito	Detalhes
Sistema Operacional	macOS 13+, Ubuntu 20+, Windows 10+ (WSL/Git Bash)
Terminal	Terminal nativo (macOS), PowerShell (Windows), qualquer terminal Linux
Git	Recomendado (obrigatório no Windows — Git for Windows)
Node.js	Apenas para instalação via npm (18+). Instalador nativo dispensa Node.js
Conta Claude	Claude Pro (\$20/mês), Max (\$100-200/mês), ou créditos API

Planos e Preços

Plano	Preço/mês	Uso Recomendado
Claude Pro	US\$ 20	Uso leve a moderado, ideal para começar
Claude Max 5x	US\$ 100	Uso regular em projetos médios
Claude Max 20x	US\$ 200	Uso pesado diário, grandes refatorações
API (Console)	Pay-per-use	Pagar por token consumido

5. Instalação no Terminal (CLI)

Caminho Recomendado: Usar com Cursor

1. Baixe o Cursor em cursor.com
2. Abra sua pasta de projeto no Cursor
3. Abra o terminal dentro do Cursor (View → Terminal)
4. Digite `claude` e pressione Enter
5. Autentique via popup do navegador

Alternativa: Terminal Direto

macOS / Linux

```
curl -fsSL https://claude.ai/install.sh | sh
```

Windows (PowerShell)

```
irm https://claude.ai/install.ps1 | iex
```

Via Homebrew (macOS)

```
brew install claude-code
```

Via WinGet (Windows)

```
winget install Anthropic.ClaudeCode
```

Depois da instalação, navegue até sua pasta de projeto e execute:

```
cd ~/seu-projeto-folder  
claude
```

Verificação

```
claude --version
```

⚠ Método npm (Legado)

Tutoriais antigos recomendam `'npm install -g @anthropic-ai/claude-code'`. Ainda funciona, mas não é mais recomendado. Prefira o instalador nativo.

6. Integração com VS Code

A extensão oficial traz toda a experiência do Claude dentro do editor, com diffs inline, @-mentions, Plan Mode e histórico.

Instalação da Extensão

1. Abra o VS Code
2. Vá em Extensions (Cmd+Shift+X / Ctrl+Shift+X)
3. Pesquise "Claude Code"
4. Instale a extensão oficial da Anthropic
5. Se não aparecer, reinicie ou rode "Developer: Reload Window"

Onde Encontrar o Claude

- Ícone Spark no canto superior direito (aparece com arquivo aberto)
- Activity Bar (sidebar esquerda) — abre lista de sessões
- Command Palette (Cmd+Shift+P / Ctrl+Shift+P) → "Claude Code"

Checkpoints (Exclusivo da Extensão)

Rastreiam edições do Claude e permitem voltar a estados anteriores:

- **Fork conversation from here** — novo branch de conversa mantendo mudanças
- **Rewind code to here** — reverte mudanças no código até este ponto
- **Fork + Rewind** — novo branch + reverte código



CLI vs Extensão

Algumas funcionalidades como MCP servers, atalho ! para bash e certos slash commands são exclusivos da CLI. Use 'claude' no terminal integrado do VS Code quando precisar.

7. Integração com Cursor

O Cursor é um fork do VS Code com IA nativa. O Claude Code não detecta o Cursor automaticamente, mas a instalação manual funciona perfeitamente.

Método 1: Via Linha de Comando

1. Certifique-se de que o Claude Code CLI já está instalado
2. Execute no terminal:

```
cursor --install-extension  
~/.claude/local/node_modules/@anthropic-ai/claude-code/vendor/claude-code.vsi  
x
```

3. Reinicie o Cursor

Método 2: Arrastar VSIX Manualmente

1. Localize o arquivo .vsix:

```
# macOS / Linux:  
~/.claude/local/node_modules/@anthropic-ai/claude-code/vendor/claude-code.vsi  
x
```

```
# Windows:  
%USERPROFILE%\claude\local\node_modules\@anthropic-ai\claude-code\vendor\cl  
aude-code.vsix
```

2. No Cursor: Extensions → arraste o .vsix para o painel
3. OU Command Palette → "Extensions: Install from VSIX..."
4. Feche e reabra o Cursor

Claude Code + Cursor Agent Juntos

- Cursor Agent para scaffolding rápido, autocompletar, edições inline
- Claude Code para refatorações complexas, análise profunda e Git
- Use branches separados para evitar conflitos

⚠ WSL no Windows

Se Claude Code roda no WSL e Cursor no Windows nativo, copie o .vsix para uma pasta Windows (ex: Downloads) e instale a partir daí.

8. Comandos Essenciais e Atalhos

Comandos dão controle preciso. Em vez de instruções longas, um comando dispara ações complexas.

Slash Commands (digite / no Claude Code)

Comando	O que faz
/help	Mostra todos os comandos disponíveis
/clear	Reseta o contexto (usar entre tarefas)
/compact	Comprime a conversa para economizar tokens
/model	Alterna entre Opus, Sonnet e Haiku
/mcp	Verifica conexões de MCP servers
/doctor	Diagnostica problemas de instalação
/config	Abre configurações
/plan	Entra no Plan Mode
/init	Gera o CLAUDE.md automaticamente
/usage	Ver consumo do plano

Referências de Arquivo (digite @ para referenciar)

Sintaxe	Exemplo
@filename	@report.csv
@folder/	@data/
Tab key	Autocomplete de caminhos

Atalhos de Teclado

Ação	Tecla
Cancelar/Parar	Esc
Rewind checkpoint	Esc duas vezes
Colar imagem	Ctrl+V (não Cmd+V no Mac)
Rodar shell direto	Começar com !
Alternar foco editor/Claude	Cmd+Esc / Ctrl+Esc
Nova linha sem enviar	Shift+Enter

9. Skills e CLAUDE.md

Claude Skills (Automações Reusáveis)

Skills são pacotes de instruções por tarefa que o Claude carrega automaticamente quando relevante.

Estrutura de Pastas

```
~/ .claude/skills/  
  linkedin-writer/  
    SKILL.md  
  prd-generator/  
    SKILL.md  
  data-cleaner/  
    SKILL.md
```

Skills Nativas

Skill	Função
docx	Documentos Word
xlsx	Planilhas
pptx	Apresentações
pdf	Processar PDFs

CLAUDE.md (Memória do Projeto)

Arquivo markdown que dá ao Claude contexto permanente sobre seu projeto.

Gerando Automaticamente

```
> /init
```

Exemplo

```
# Projeto: Marketing Dashboard  
  
## Key Commands  
- npm run build  
- npm test  
  
## Style Guide  
- Use TypeScript  
- Follow existing patterns  
  
## Important Context
```

- Main data source: /data/analytics.csv
- Deploy target: vercel

Onde Colocar

- **Global:** ~/.claude/CLAUDE.md — aplica a todos os projetos
- **Projeto:** ./CLAUDE.md — específico do projeto



Dica Pro

Quanto mais detalhado o CLAUDE.md, melhores os resultados. Inclua stack, convenções, comandos de build/test e preferências da equipe.

10. Plan Mode — Planejamento Inteligente

Em vez de sair executando, o Claude analisa, planeja e espera aprovação.

1. Ative com `/plan` ou pelo menu (/)
2. Descreva a tarefa
3. Claude analisa arquivos e traça dependências
4. Apresenta plano passo a passo
5. Você aprova, ajusta ou rejeita

Quando Usar

- Refatorações multi-arquivo
- Features novas complexas
- Codebases desconhecidos
- Aprendizado — a explicação ajuda a entender o projeto

Exemplo

```
> /plan  
> Refatorar autenticação para JWT.
```

Claude responde:

1. Analisar `auth/session.py` e `middleware.py`
2. Criar `auth/jwt_handler.py`
3. Atualizar rotas
4. Modificar testes
5. Atualizar `CLAUDE.md`

11. Técnicas de Prompting

Técnica	Quando Usar
Ser Específico	Sempre. "Limpe CSV → Remova linhas onde coluna B está vazia, deduplique por email"
Dar Exemplos	Quando formato importa. Mostre 1-2 exemplos do resultado esperado
Passos em Cadeia	"Primeiro analise, depois resuma, então crie action items"
Controle de Qualidade	"Max 500 palavras" ou "Use dados de 2024"
Assign Roles	"Aja como senior backend engineer revisando este módulo"
Usar /clear	Resete entre tarefas não relacionadas

Pro Pattern: Checkpoint + Iterate

1. Peça um plano primeiro: "crie um plano para..."
2. Revise o plano antes de executar
3. Mande executar com checkpoints
4. Se algo sair errado, Rewind (Esc twice)
5. Itere com feedback específico

12. Modelos Disponíveis

Modelo	Melhor Para	Contexto
Sonnet 4.6	80% do trabalho: funções, bugs, features, testes	200K tokens
Opus 4.6	20% complexo: refatorações, arquitetura, Agent Teams	200K (1M beta)

> /model

> Selecione: claude-sonnet-4-6 ou claude-opus-4-6



Estratégia

Comece com Sonnet. Escale para Opus apenas para raciocínio profundo ou contextos muito grandes.

13. Conecte Tudo com MCP

MCP (Model Context Protocol) é um padrão aberto que conecta Claude às suas ferramentas. Pense como USB-C para IA: um protocolo, centenas de conexões.

Categorias de MCP Servers

Categoria	Ferramentas
Developer Tools	GitHub, Sentry, Linear, GitLab
Productivity	Notion, Slack, Google Drive, Asana
Data & Research	Perplexity, PostgreSQL, Brave, Filesystem
Communication	Gmail, Typefully, Buffer, Discord

Comando Rápido

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

```
claude mcp add nome-server -- comando args
```

```
claude mcp list
```



MCP + n8n

Crie MCP servers customizados via n8n webhooks. Conecte Claude Code com qualquer workflow de automação existente.

14. Workflow Combinado: Terminal + IDE

Tarefa	Onde	Por quê
Codificação diária	Extensão VS Code/Cursor	Diffs visuais, @-mentions, checkpoints
Refatorações pesadas	Terminal CLI	Controle total, contexto máximo
Operações Git	Terminal CLI	Commits, PRs, branches automatizados
Debug de builds	Terminal CLI	Acesso a logs e stack traces
Brainstorming	Web/Desktop App	Interface leve, qualquer lugar

Via pipe (CLI)

```
claude -p "Rode testes de integração, corrija falhas e explique."
```

```
cat error.log | claude -p "Analise e sugira correções."
```

```
git diff main..feat/nova | claude -p "Faça code review."
```

15. Dicas e Boas Práticas

Para Resultados Melhores

- Sempre comece com CLAUDE.md configurado
- Prompts específicos: "Corrija NullPointerException em handleUserInput()" > "Corrija o bug"
- Use /plan antes de mudanças grandes
- Limite contexto com @-mentions
- Sonnet para 80%, Opus para 20% complexos
- Revise antes de aprovar
- Use /clear entre tarefas

Erros Comuns

Erro	Solução
Prompts vagos	Seja específico sobre problema e resultado
Projeto inteiro sem foco	@-mentions para focar arquivos relevantes
Sem CLAUDE.md	/init no início de cada projeto
Ignorar Plan Mode	/plan para tarefas com 3+ arquivos
Opus para tudo	Sonnet basta para a maioria
Dois AIs na mesma branch	Branches separados

16. Comparação: Claude Code vs Cursor AI

Aspecto	Claude Code	Cursor AI
Abordagem	Agente autônomo (terminal-first)	IDE com IA (VS Code fork)
Modelos	Exclusivamente Claude	Multi-modelo (Claude, GPT-5, Gemini)
Contexto Real	200K tokens (1M beta)	70K-120K na prática
MCP Servers	200+ tools, sem limite	Até 40 ferramentas
Autonomia	Alta — multi-step autônomo	Média — mais interativo
Git Nativo	Completo	Parcial
Skills	Sim	Não
Preço Base	US\$ 20/mês	US\$ 20/mês
Uso Pesado	US\$ 100-200/mês	US\$ 60-200/mês
Melhor Para	Refatorações, CI/CD, automação	Edição rápida, tab completion

Veredicto

Não é "um ou outro". Use Cursor para editar rápido, Claude Code para tarefas pesadas que exigem profundidade e autonomia.

17. Estrutura de Arquivos do Projeto

Todo projeto Claude Code segue uma estrutura de arquivos específica. Conhecer essa estrutura é fundamental para configurar corretamente o ambiente.

```
your_project/
├── CLAUDE.md                # Memória do projeto
├── .claude/
│   ├── settings.json        # Config compartilhada (vai pro Git)
│   ├── settings.local.json  # Config local (não vai pro Git)
│   ├── skills/
│   │   ├── code-review/
│   │   │   └── SKILL.md
│   │   ├── testing/
│   │   │   └── SKILL.md
│   │   └── helpers.py
│   ├── commands/
│   │   └── deploy.md
│   └── agents/
│       └── security-reviewer.md
├── arc/
└── .gitignore
```

O que cada arquivo faz

Arquivo	Função
CLAUDE.md	Memória persistente do projeto (contexto, stack, regras)
settings.json	Configurações compartilhadas (MCP, permissões, hooks)
settings.local.json	Configurações pessoais (não vai pro Git)
skills/	Skills customizadas por projeto
commands/	Comandos reusáveis (.md com instruções)
agents/	Sub-agentes com contexto próprio



Dica para Equipes

Faça commit do .claude/settings.json e da pasta skills/ no Git para compartilhar configurações com o time. Mantenha settings.local.json no .gitignore.

18. Hierarquia de Memória do CLAUDE.md

O Claude Code carrega arquivos CLAUDE.md de múltiplos níveis, criando uma hierarquia de contexto do global ao específico:

Nível	Local	Escopo
Global	~/claude/CLAUDE.md	Aplica a todos os projetos
Monorepo Root	~/CLAUDE.md	Pai do monorepo
Projeto	./CLAUDE.md	Compartilhado via Git com o time
Subpasta	./frontend/CLAUDE.md	Contexto limitado ao subdiretório

Regras Importantes

- **Manter cada arquivo com menos de 200 linhas** — memória concisa é mais eficaz
- **Arquivos de subpastas acrescentam contexto** — não substituem os níveis acima
- **Nunca sobrescreva o contexto pai** — cada nível é aditivo

Boas Práticas do CLAUDE.md

O CLAUDE.md precisa responder 3 perguntas fundamentais: O QUÊ, POR QUÊ e COMO.

O QUÊ (WHAT)	POR QUÊ (WHY)	COMO (HOW)
• Tech stack • Mapa de diretórios • Arquitetura	• Propósito de cada módulo • Decisões de design • Workflows	• Comandos build/test/lint • Gotchas e armadilhas • Regras de workflow

Checklist de Boas Práticas

- Rode /init primeiro e depois refine o output
- Seja específico nas instruções
- Adicione gotchas que o Claude não consegue inferir
- Referencie docs com @filename
- Adicione regras de workflow

- Mantenha a memória concisa (< 200 linhas)
- Faça commit no Git para compartilhar com o time

19. Skills Avançadas — O Superpoder

Skills são guias em markdown que o Claude auto-invoca via linguagem natural. São o recurso mais poderoso para criar automações reusáveis.

Skill de Projeto vs Skill Pessoal

Tipo	Local	Escopo
Skill de Projeto	.claude/skills/<nome>/SKILL.md	Compartilhada via Git com o time
Skill Pessoal	~/.claude/skills/<nome>/SKILL.md	Apenas para você, em todos os projetos

Anatomia de uma SKILL.md

O campo description é crítico — é ele que determina quando o Claude ativa a skill automaticamente.

```
---
name: testing patterns
description: Jest testing patterns
allowed tools: Read, Grep, Glob
---

# Testing Patterns
Use describe + it + AAA pattern
Use factory mocks
Always write tests before implementation
```

Ideias de Skills para Engenheiros

Skill	Uso
code-review	Padrões e checklist de code review
testing-patterns	Convenções de teste (Jest, pytest, etc.)
commit-messages	Conventional Commits + regras do time
docker-deploy	Dockerfile + docker-compose patterns
codebase-visualizer	Mapear e documentar arquitetura
api-design	Padrões REST/GraphQL + naming conventions

Pro Tip

O campo 'description' na frontmatter da SKILL.md é o que faz a auto-ativação funcionar. Seja específico e descritivo. Ex: 'Jest testing patterns for React components with RTL' em vez de apenas 'testing'.

20. Hooks — Automação Determinística

Hooks são callbacks determinísticos que rodam em momentos específicos da execução do Claude. Diferente de skills (que são probabilísticas), hooks SEMPRE executam quando configurados.

Tipos de Hooks

Hook	Quando Executa	Uso Comum
PreToolUse	Antes de usar uma ferramenta	Validação de segurança, bloqueio de comandos perigosos
PostToolUse	Depois de usar uma ferramenta	Logging, notificações, linting automático
Notification	Em eventos específicos	Alertas, integrações externas

Exemplo de Configuração

Configure hooks no settings.json do projeto:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/sec.sh",
            "timeout": 5
          }
        ]
      }
    ]
  }
}
```

Códigos de Saída

Código	Resultado
0	Permitir — a ferramenta executa normalmente

Código	Resultado
2	Bloquear — a ferramenta é impedida de executar
Qualquer outro	Erro — o Claude decide o que fazer



Hooks vs Skills

Skills são probabilísticas (o Claude decide quando usar). Hooks são determinísticos (SEMPRE executam). Use hooks para segurança e compliance, skills para padrões de código.

21. Permissões e Segurança

O Claude Code permite controle granular sobre o que o agente pode e não pode fazer, usando padrões allow/deny no settings.json.

Exemplo de Configuração

```
{
  "permissions": {
    "allow": [
      "Read:*",
      "Bash:git:*",
      "Write:*:*md"
    ],
    "deny": [
      "Read:env:*",
      "Bash:sudo:*"
    ]
  }
}
```

Padrões Comuns de Permissão

Padrão	Significa
Read:*	Pode ler qualquer arquivo
Bash:git:*	Pode executar qualquer comando git
Write:*:*md	Pode escrever apenas arquivos .md
Read:env:* (deny)	Não pode ler variáveis de ambiente
Bash:sudo:* (deny)	Não pode executar comandos com sudo

Segurança para Times

Combine permissões no settings.json (compartilhado via Git) com hooks PreToolUse para criar guard rails robustos. Bloqueie acesso a .env, secrets e comandos destrutivos.

22. Arquitetura de 4 Camadas

O Claude Code opera com uma arquitetura em 4 camadas que se complementam. Entender essa hierarquia é a chave para configurações avançadas.

L1 — CLAUDE.md — Contexto persistente e regras do projeto
L2 — Skills — Pacotes de conhecimento auto-invocados
L3 — Hooks — Guard rails de segurança e automação
L4 — Agents — Sub-agentes com contexto próprio

Como as Camadas Interagem

- **L1 (CLAUDE.md)** fornece contexto base que todas as outras camadas utilizam
- **L2 (Skills)** são ativadas automaticamente baseado no contexto da tarefa
- **L3 (Hooks)** validam e filtram antes/depois de cada ação do agente
- **L4 (Agents)** operam em paralelo, cada um com suas próprias skills e contexto

23. Workflow Diário Recomendado

Este é o padrão de trabalho diário que maximiza produtividade com Claude Code:

1	<code>cd project && claude</code> — Inicie na pasta do projeto
2	<code>Shift+Tab+Tab</code> → <code>Plan Mode</code> — Entre no modo de planejamento
3	<code>Descreva a feature intent</code> — Explique o que quer construir
4	<code>Shift+Tab</code> → <code>Auto Accept</code> — Ative aceitação automática quando confia
5	<code>/compact</code> — Compacte contexto periodicamente
6	<code>Esc Esc</code> → <code>Rewind</code> — Volte atrás se algo sair errado
7	<code>Commit frequently</code> — Salve progresso com commits frequentes
8	<code>Nova sessão por feature</code> — Inicie sessão limpa para cada feature

Quick Reference Expandido

Comando / Atalho	Função
<code>/init</code>	Gerar CLAUDE.md
<code>/doccat</code>	Checar instalação
<code>/compact</code>	Comprimir contexto
<code>Shift + Tab</code>	Mudar modo de aceitação
<code>Tab</code>	Toggle extended thinking
<code>Esc Esc</code>	Menu de rewind
<code>/model</code>	Trocar modelo
<code>/plan</code>	Entrar em Plan Mode
<code>!</code>	Rodar comando shell direto

24. Recursos e Links Úteis

Recurso	Link
Documentação Oficial Claude Code	code.claude.com/docs
Quickstart	code.claude.com/docs/en/quickstart
VS Code Integration	code.claude.com/docs/en/vs-code
Setup Oficial	code.claude.com/docs/en/setup
Guia Básico	docs.claude.com/en/docs/claude-code/overview
Guia Avançado	docs.claude.com/en/docs/claude-code
Prompt Engineering	docs.claude.com/en/docs/build-with-claude/prompt-engineering
Anthropic MCP Servers	github.com/anthropics/mcp-servers
Coleção MCP.so	mcp.so
Suporte Claude.ai	support.claude.com
Notícias Anthropic	anthropic.com/news

flowgrammers

O Maior Ecossistema de Educação em Inteligência Artificial e Automações do Brasil

flowgrammers.com.br

Da comunidade ativa aos workshops enterprise, o caminho completo para dominar automação e IA.

© 2026 Flowgrammers AI Academy. Todos os direitos reservados.